

Библиотека драйвера Modbus RTU для ПЛК Coolmay FX3G

Терминология

- Регистр — область памяти в адресном пространстве Modbus.

Требования

Для работы библиотеки необходимо установить следующие библиотеки:

- UtilsV4.sul
- TimeControV2.sul

[!IMPORTANT]

- Библиотека требует настройки TCO Ticker в библиотеке Time Control 50. Настройте его в первую очередь.
- Работает только с GXW2 v1.91. Установите обновления из папки [coolmay/soft](#).

Список изменений

v1.5 [09.04.25]

- добавлено — свойство `xWriteOnce` для канала

v1.4 [09.04.25]

- добавлено — очистка всех регистров, которые будут использоваться для хранения данных, перед первым циклом чтения.
- добавлено — свойство `xDone` для каналов. Формирует импульс при завершении каналом одного цикла.

v1.3 [28.02.25]

- исправлено — ошибка, при которой соединение не восстанавливалось после тайм-аута. Спасибо @Alex_315.

v1.2 [12.12.24]

- изменено — свойство канала `iReg` теперь имеет тип `Word[Unsigned]/Bit String[16-bit]`. Позволяет задавать адрес регистра больше 32000.
- добавлено — константы `MB_PORT_2`, `MB_PORT_3`, `MB_PORT_CAN` и `MB_PORT_TCP` для свойства канала `iPort`.

v1.1 [29.09.24]

- изменены регистры R на D, так как некоторые панели (например, OP320) не поддерживают R по протоколу Modbus.

v1.0 [22.09.24]

- Первая версия библиотеки со всеми запланированными возможностями.

Описание

Библиотека позволяет настроить ПЛК Coolmay FX3G как Modbus Slave или Modbus Master (на портах 2 и 3) для чтения и записи данных устройств Modbus по протоколу RTU. Обеспечивает простой интерфейс для конфигурации мастера.

Панельные контроллеры Coolmay (ПЛК + HMI) имеют два порта RS485: порт 2 выведен на клеммный разъём, порт 3 — на разъём DB9. ПЛК серии L02 также имеют два порта RS485, оба выведены на клеммные разъёмы.

MB_PORT_SETTINGS

Функция формирует переменную с корректными битами для инициализации порта в режиме мастера или слейва.

Переменная	Область	Тип	Описание
Parity	INPUT	(Word[Signed])	Используйте константы MB_PARITY_NONE, MB_PARITY_ODD или MB_PARITY_EVEN
StopBit	INPUT	(Word[Signed])	Используйте константы MB_STOPBIT_1 или MB_STOPBIT_2
Baudrate	INPUT	(Word[Signed])	Используйте константы MB_BPS_600, MB_BPS_1200, MB_BPS_2400, MB_BPS_4800, MB_BPS_9600, MB_BPS_19200, MB_BPS_38400, MB_BPS_57600 или MB_BPS_115200

Объявите локальную переменную типа Unsigned Double Word в своей программе, например PortSettings, затем вызовите функцию.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
```

Modbus Slave

MB_SLAVE_INIT_PORT2, MB_SLAVE_INIT_PORT3

Функция инициализирует Modbus-слейв на порту 2 или порту 3 ПЛК.

Переменная	Область	Тип	Описание
<code>xInit</code>	INPUT	BIT	Сигнал инициализации. Инициализация происходит по каждому фронту данного параметра.
<code>iAddress</code>	INPUT	(Word[Signed])	Адрес данного ПЛК в сети Modbus
<code>PortSettings</code>	INPUT	(Double word[Signed])	Результат функции MB_PORT_SETTINGS

Пример настройки слейва с адресом 1 на порту 2.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
M0 := MB_SLAVE_INIT_PORT2(TRUE, 1, PortSettings);
```

Дальнейшая настройка не требуется — ПЛК работает как слейв на порту 2. Тот же подход применяется для порта 3. `M0` нигде не используется — это формальное требование синтаксиса вызова функции; присваивание должно иметь левую часть.

Modbus Master

MB_MASTER_INIT_PORT2, MB_MASTER_INIT_PORT3

Функции инициализируют Modbus-мастер на порту 2 (клеммный разъём) или порту 3 (DB9) соответственно. Обе принимают одинаковые параметры.

Переменная	Область	Тип	Описание
<code>xInit</code>	INPUT	BIT	Сигнал инициализации. Инициализация происходит по каждому фронту данного параметра.
<code>PortSettings</code>	INPUT	(Double word[Signed])	Результат функции MB_PORT_SETTINGS

Пример настройки мастера на порту 2.

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
M0 := MB_MASTER_INIT_PORT2(TRUE, PortSettings);
```

MB_PROCESS_50

Функциональный блок выполняет операции чтения и записи для всех настроенных каналов.

Перед использованием установите библиотеку TimeControl50 и настройте `TCO_TICKER_50`. Этот тикер инкрементируется с интервалом 50 мс.

Переменная	Область	Тип	Описание
<code>mb_xEnable</code>	INPUT	BIT	Запуск обработки каналов.
<code>mb_iBuffer</code>	INPUT	(Word[Signed])	Номер устройства D для регистров и M для катушек, используемых в качестве буфера. Например, если указано 100, то D100 будет первым устройством буфера для регистров или M100 — для катушек. Будет занято столько устройств, сколько регистров или катушек считывается из одного канала (<code>iNum</code>).
<code>mb_Timeout</code>	OUTPUT	(Word[Signed])	Номер устройства, для которого истёк тайм-аут.

Глобальный массив `MB_CHANNELS` из 30 элементов структуры `MB_REG_50` объявлен в библиотеке. Объявлять его вручную не требуется. Массив определяет список обрабатываемых каналов. Каждый канал может читать или записывать до 125 регистров. Массив необходимо настроить один раз, обычно при запуске ПЛК с использованием флага `M8002`.

В следующей таблице перечислены параметры структуры `MB_REG_50`.

Переменная	Тип	Описание
<code>iDDevNum</code>	(Word[Signed])	Номер устройства D для хранения результата чтения регистров и M для катушек. Например, если задано K200 , то результат чтения будет находиться в D200 (регистры) или M200 (катушки).
<code>iNum</code>	(Word[Signed])	Количество считываемых регистров или катушек. По умолчанию 1.
<code>iReg</code>	Word[Unsigned]/Bit String[16-bit]	Начальный адрес регистра Modbus (holding/input) в десятичном формате.
<code>iRF</code>	Word[Unsigned]/Bit String[16-bit]	Функция чтения Modbus. По умолчанию H3 .
<code>iWF</code>	Word[Unsigned]/Bit String[16-bit]	Функция записи Modbus. По умолчанию H6 .
<code>iDev</code>	Word[Unsigned]/Bit String[16-bit]	Адрес подчинённого устройства.
<code>tCycle</code>	(Word[Signed])	Интервал цикла для канала в шагах по 50 мс. Например, для чтения раз в 100 мс задайте <code>tCycle = 2</code> . Установите 0 для ручного чтения через параметр <code>xReadOnce</code> .
<code>iWR</code>	(Word[Signed])	Режим чтения/записи. По умолчанию <code>MB_READ_WRITE</code> . Можно задать <code>MB_READ</code> или <code>MB_WRITE</code> .
<code>iPort</code>	(Word[Signed])	Коммуникационный порт. Порт 2 — 0, Порт 3 — 1, CAN — 2, Ethernet Modbus TCP — 3.

Переменная	Тип	Описание
<code>xDone</code>	Bit	Цикл чтения/записи канала завершён. В основном предназначен для каналов с ручным циклом (<code>xReadOnce</code>).
<code>xEnabled</code>	Bit	Канал обрабатывается только при включённом состоянии.
<code>xReadOnce</code>	Bit	По фронту (OFF → ON) запускает однократное чтение канала. Для полностью ручного чтения установите <code>tCycle = 0</code> .
<code>xWriteOnce</code>	Bit	По фронту (OFF → ON) запускает однократную запись канала. Для полностью ручной записи установите <code>tCycle = 0</code> .
<code>xWriteOnChange</code>	Bit	При <code>TRUE</code> изменения немедленно записываются в подчинённое устройство. При <code>FALSE</code> запись происходит только по истечении интервала <code>tCycle</code> .

Поддерживаемые функции

- `H1` — Read Coils (чтение катушек)
- `H2` — Read Discrete Inputs (чтение дискретных входов)
- `H3` — Read Holding Register (чтение регистров хранения)
- `H4` — Read Input Register (чтение входных регистров)
- `H5` — Write Single Coil (запись одной катушки)
- `H6` — Write Single Register (запись одного регистра)
- `HF` — Write Multiple Coils (запись нескольких катушек)
- `H10` — Write Multiple Registers (запись нескольких регистров)

Поддерживаемые порты

- `MB_PORT_2` — RS485, первый порт A,B
- `MB_PORT_3` — RS485, второй порт A1,B1
- `MB_PORT_CAN` — порт CAN H,L
- `MB_PORT_TCP` — порт Ethernet

`iDDevNum`

Каждый канал резервирует удвоенное количество устройств относительно значения `iNum`. Если `iNum = 1` и `iDDevNum = 200`, то `D200` хранит результат, а `D201` используется для внутренних нужд. Если `iNum = 5`, то `D200–D204` хранят результаты, а `D205–D209` используются для внутренних расчётов. Это необходимо учитывать при настройке каналов.

Рассмотрим следующую конфигурацию:

```
MB_CHANNELS[0].iDDevNum := 600;
MB_CHANNELS[0].iNum := 3;
```

```
MB_CHANNELS[1].iDDevNum := 603;  
MB_CHANNELS[1].iNum := 1;
```

На первый взгляд конфигурация кажется корректной, поскольку второй канал использует следующий свободный номер устройства, однако она не учитывает устройства для внутренних расчётов и потому работать не будет.

При чтении 3 регистров в **D600** следующий доступный номер устройства — **D606**.

iWF

Для записи регистров поддерживаются две функции: **H6** выполняет запись одного регистра, **H10** — запись группы регистров. Если на канале с несколькими регистрами задана **H6**, при обнаружении изменения записывается только изменённый регистр. Если задана **H10**, изменение любого регистра в группе вызывает один запрос, обновляющий все регистры группы. Для значений двойного слова используйте **H10**. Если канал содержит несколько независимых однорегистровых переменных, используйте **H6**. Для одноканального канала также используйте **H6**.

То же правило действует при работе с катушками. Используйте **H5** даже на каналах с несколькими катушками.

Пример

Пример программы.

```
IF M8002 THEN  
  
    (* Количество последовательных тайм-аутов, после которых канал помечается как  
    приостановленный. По умолчанию: 2 *)  
    MB_TIMEOUT_COUNT := 2;  
    (* Периодичность повтора приостановленного канала в шагах 50 мс. По умолчанию:  
    80 (4 секунды) *)  
    MB_SUSPEND_RETRY := 80;  
    (* Время отсутствия ответа, считающееся тайм-аутом. По умолчанию: 4 (200  
    миллисекунд) *)  
    MB_TIMEOUT_TIME := 4;  
  
    PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);  
    (* Multi-master доступен для обоих портов на L02 *)  
    M0 := MB_MASTER_INIT_PORT2(TRUE, PortSettings);  
    M0 := MB_MASTER_INIT_PORT3(TRUE, PortSettings);  
  
    MB_CHANNELS[0].xEnabled := TRUE;  
    MB_CHANNELS[0].iDDevNum := 600;  
    MB_CHANNELS[0].iNum := 3;  
    MB_CHANNELS[0].iReg := K16384;  
    MB_CHANNELS[0].iRF := H3;  
    MB_CHANNELS[0].iWF := H6;
```

```
MB_CHANNELS[0].iDev := H1;
MB_CHANNELS[0].tCycle := 20; (* 20 * 50ms = 1000ms или 1 секунда *)
MB_CHANNELS[0].xWriteOnChange := TRUE;
MB_CHANNELS[0].iWR := MB_READ_WRITE;
MB_CHANNELS[0].iPort := MB_PORT_2;

(* Минимальная настройка для подключения устройств через порт 3 *)
MB_CHANNELS[1].xEnabled := TRUE;
MB_CHANNELS[1].iDDevNum := 610;
MB_CHANNELS[1].iReg := K16385;
MB_CHANNELS[1].iDev := K16;
MB_CHANNELS[1].tCycle := 4;
MB_CHANNELS[1].iWR := MB_READ;
MB_CHANNELS[1].iPort := MB_PORT_3;

(* Ручной цикл чтения и записи группы из 3 регистров *)
MB_CHANNELS[2].xEnabled := TRUE;
MB_CHANNELS[2].iNum := 3;
MB_CHANNELS[2].iDev := 1;
MB_CHANNELS[2].iPort := MB_PORT_3;
MB_CHANNELS[2].iDDevNum := 10;
MB_CHANNELS[2].iReg := H1000;
MB_CHANNELS[2].iRF := H3;
MB_CHANNELS[2].iWR := MB_READ;
MB_CHANNELS[2].tCycle := 0;
END_IF;

fbMbProcess(mb_xEnable := TRUE, mb_iBuffer := 100);

(* однократное ручное чтение канала 2 *)
IF M1 THEN
    MB_CHANNELS[2].xReadOnce := TRUE;
    IF MB_CHANNELS[2].xDone = TRUE THEN
        M1 := FALSE;
        MB_CHANNELS[2].xReadOnce := FALSE;
    END_IF;
END_IF;

(* однократная ручная запись канала 2 *)
IF M2 THEN
    D10 := 100;
    D11 := 100;
    D12 := 100;
    MB_CHANNELS[2].xWriteOnce := TRUE;
    IF MB_CHANNELS[2].xDone = TRUE THEN
        M2 := FALSE;
        MB_CHANNELS[2].xWriteOnce := FALSE;
    END_IF;
END_IF;

(* Альтернативный способ проверки успешности записи
при ручной записи канала 2 *)
IF M2 THEN
```

```
D10 := 200;  
D11 := 200;  
D12 := 200;  
MB_CHANNELS[2].xWriteOnce := TRUE;  
IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN  
    M2 := FALSE;  
    MB_CHANNELS[2].xWriteOnce := FALSE;  
END_IF;  
END_IF;
```

После настройки **D600**, **D601** и **D602** будут содержать результаты первого канала от устройства с адресом 1, обновляемые каждую секунду. При изменении значения подчинённое устройство обновляется немедленно.

D610 будет содержать результат второго канала. Поскольку канал работает только на чтение (**MB_READ**), любое локальное изменение **D610** будет перезаписано следующим чтением от подчинённого устройства через 200 мс.

Альтернативное подтверждение записи

Логика условия **IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN** состоит в следующем: как описано в разделе **iDDevNum**, каждый канал резервирует удвоенное количество устройств. При **iDDevNum = 10** результат сохраняется начиная с **D10**. При чтении одного регистра **D10** хранит фактическое значение, а **D11** служит буфером отслеживания изменений. При чтении 3 регистров **D10–D12** хранят фактические значения, а **D13–D15** — буфер отслеживания изменений, который обновляется до текущих значений при успешной записи. Если значения совпадают с соответствующими буферами — запись выполнена успешно.

Это особенно полезно при использовании **xWriteOnChange** (вместо **xWriteOnce**) с каналом из 10 регистров и функцией записи **H6**. Если изменены 2 регистра, **H6** записывает их по одному, устанавливая **xDone** при каждой успешной записи. Подсчёт 2 импульсов **xDone** может быстро усложниться. Для сравнения, проверка вида **IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN** подтверждает успешное обновление всех изменённых регистров одной операцией.

Тайм-ауты

Библиотека реализует механизм приостановки каналов. Если канал не отвечает в течение **MB_TIMEOUT_COUNT** последовательных попыток, он помечается как приостановленный.

Приостановленные каналы опрашиваются один раз за интервал **MB_SUSPEND_RETRY**. Как только ответ получен, отметка о приостановке снимается, и канал возобновляет нормальный цикл согласно **MB_CHANNELS[*].tCycle**.